Anshuman's Guide: Heart Votes, Calibration and Late Fusion

Your part of the heart roadmap, step by step, Monday 2 to Tuesday 17 November 2026

CS F407 Artificial Intelligence · Team CardioNary · Branch `cardio` · A companion to the heart roadmap, which stays the reference

1. Your job in one paragraph

The heart CNN listens to a recording and says how likely it is to be abnormal. Proposal Section 4.2 adds two more opinions before the heart branch gives its answer:

- **Feature vote:** simple measurements of the heartbeat (heart rate, murmur timing and loudness), which a doctor could check by ear.
- **Metadata vote:** the patient's details (age group, sex, height, weight).

Your job has four parts:

1. Build those two votes.
2. Make the CNN's probability honest (calibration).
3. Combine the three into one number for the heart, p_h (late fusion).
4. Show with numbers whether the extra votes help.

You never train a CNN. Figure 1 shows your part.

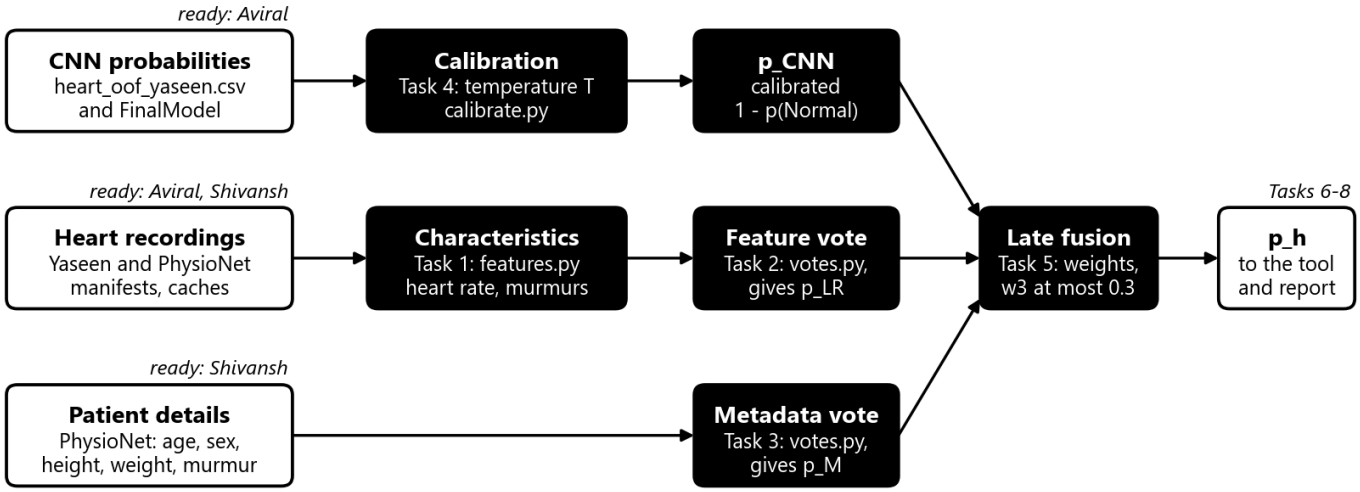


Figure 1. Your part of the heart pipeline. The white boxes are already built by Aviral and Shivansh; the black boxes are your tasks.

2. What your teammates have already done for you

What	Where	What it means for you
CNN results for every Yaseen clip, each from a model that never saw that clip	`reports/heart_oof_yaseen.csv`: 990 clips, with columns `patient_id`, `fold`, `label`, `p_Normal` to `p_MVP`	Your input for calibration and for the CNN vote. Ready to use.
The final heart model, and a helper to run it	`models/chiron_cardio_v1_float.keras`, `cardio/model/final_model.py` (`FinalModel`)	Gives CNN probabilities for any recording, such as PhysioNet.
PhysioNet 2022, organised	`data/manifests/circor_manifest.csv` and `data/cache/unseen/` (Shivansh's `cardio/data/build_unseen.py`)	One row per recording, with `patient_id`, the murmur label and the patient's details. Repeat visits of the same child are already joined (942 visits become 872 patients).
Metrics and 95% intervals	`cardio/eval/metrics.py` (Shivansh), `cardio/eval/intervals.py` (Aviral)	AUROC, calibration error and intervals are one function call each. Do not rewrite them.
Where calibration starts	`reports/heart_cv_yaseen.json`, section `with_intervals`	Five-class calibration error is 0.058 [0.053, 0.065]; the target is at most 0.05. The refer-or-not calibration error is already 0.020.
Heart-cycle and murmur measurements	`core/descriptors.py`, function `audio_descriptors`	Most of your characteristics in one call.

What	Where	What it means for you
Adaptation	`cardio/eval/real_world.py`, function `adapt_bn`	You need it in Task 6.
Timing of a whole case	`cardio/eval/edge_budget.py`	It lists "votes and fusion" as pending. Your code fills that gap.

Not done yet. Shivansh's `run_unseen.py` (the plain CNN on BMD-HS and PhysioNet) does not exist yet, so Task 5 shows how to get PhysioNet CNN probabilities yourself. The BMD-HS data is on Shivansh's laptop; ask him for it when you reach Task 5.

3. What is different from the roadmap, and why

The roadmap is still the reference. These are updates based on what Aviral and Shivansh have measured and built since it was written.

1. **Calibration goes the other way.** The roadmap expected a temperature above 1. The model is actually under-confident: its average top-class probability is 0.935 while its accuracy is 0.993, because training used label smoothing. Expect T below 1, around 0.5 on a rough check.
2. **Use Shivansh's PhysioNet manifest instead of `training_data.csv`.** It already joins repeat visits of the same child. Making your folds from its `patient_id` stops the same child appearing in both training and testing.
3. **A rule for patients with several visits.** A patient is Present if any visit says Present. Unknown visits are ignored, and patients with only Unknown visits are dropped. That gives 816 patients: 161 Present and 655 Absent.
4. **Only five patient facts go into the metadata vote:** age group, sex, height, weight and pregnancy. The manifest also holds murmur grading, timing, pitch, shape and locations; those give away the answer and must never be inputs.
5. **You are behind the original dates.** S1 was due on 30 Oct. Section 4 is a catch-up plan that still finishes by 17 Nov.
6. **New hand-off.** Save your fitted numbers in `reports/heart_fusion_params.json` (Task 7), so Aviral's tool and edge budget can use them.
7. **The clinic recordings stay on Aviral's laptop.** Write `features.py` so it also accepts a folder; Aviral runs it there if the clinic recordings are needed.

4. Your plan, day by day

Dates	Task	Output	Done when
Mon 2 to Tue 3 Nov	0. Set up and get the data	Data, caches and model files on your laptop	Tests pass; both manifests show no change
Tue 3 to Wed 4 Nov	1. Characteristics	`reports/heart_features_yaseen.csv`, `reports/heart_features_physionet.csv`	1,000 Yaseen rows; median heart rate about 75 bpm
Wed 4 Nov	2. Feature vote	`reports/heart_plr_yaseen.csv`	Out-of-fold p_{LR} for every clip; AUROC recorded
Thu 5 Nov	3. Metadata vote	`cardio/data/circor_folds.csv`, `reports/heart_pm_physionet.csv`	Out-of-fold p_M for 816 patients; AUROC recorded
Thu 5 to Fri 6 Nov	4. Calibration	`reports/heart_calibration.json`	T below 1; five-class calibration error at most 0.05
Sat 7 to Mon 9 Nov	5. Late fusion and the two comparisons	`reports/heart_fusion.json`	Both comparisons with 95% intervals (milestone M2)
Tue 10 Nov	6. Add the adapted model	`reports/heart_fusion.json` updated	The full branch is in the comparison (M3)
Wed 11 to Thu 12 Nov	7. Hand-off and `run_all.py`	`reports/heart_fusion_params.json`, `cardio/run_all.py`	Aviral's tool loads your numbers; `run_all.py` re-creates every file (M4, Fri 13)
Sat 14 to Tue 17 Nov	8. Report sections	Report text, experiment notes	Section 4.2 and calibration written (M5)

Push your work at the end of each task, so nobody waits on your laptop.

5. Words you will meet

Word	Plain meaning
Fold	One fifth of the data. With 5 folds you train on four and test on the fifth, five times over.
Out-of-fold	A prediction made by a model that never saw that example. Calibration and fusion may only use these.
Logistic regression	A simple model that turns several numbers into one probability (`sklearn LogisticRegression`).
Standardise	Shift and scale each number to mean 0 and spread 1 (`StandardScaler`), learned on training folds only.
Median filling	Replace a missing number with the median of the training folds (`SimpleImputer`).

Word	Plain meaning
AUROC	How well a probability ranks abnormal above normal: 0.5 is guessing and 1.0 is perfect.
Calibration	Making "80% sure" mean right 80% of the time.
Temperature T	One number that makes the CNN more or less confident: below 1 sharpens, above 1 softens. It never changes which class wins.
Calibration error (ECE)	The average gap between how sure the model is and how often it is right; lower is better.
Late fusion	Adding the three votes' probabilities with weights that sum to 1.
Log-loss	A score for how good probabilities are; lower is better. Used to choose the weights.
Bootstrap interval	Resample the patients 1,000 times and recompute; the middle 95% of the results is the interval.
Leakage	A test example, or the same patient, also used in training. It makes results look better than they are.

6. Task 0: Set up and get the data (about an hour, plus downloads)

1. Get the code, make your branch and run the tests:

```
git checkout cardio
git pull
git checkout -b cardio-anshuman-votes
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
python -m pytest tests/
```

2. Get the final heart model files. They are in Git on `main`, while `cardio` ignores the `models/` folder, so restore them into your folder:

```
git fetch origin
git restore --source=origin/main --worktree -- models/chiron_cardio_v1_float.keras
models/chiron_cardio_v1_int8.tflite
```

3. Get Yaseen and build its cache. The second-last line must print nothing, which proves your copy matches the committed folds. In PowerShell, set `$env:PYTHONUTF8 = '1'` first if you see an encoding error, because some Yaseen folder names contain Korean characters.

```
python core/download_datasets.py --dataset yaseen
python -m core.build_cardio_manifest
git diff --stat data/manifests/yaseen_manifest.csv
python -m core.preprocess --dataset yaseen
```

4. Get PhysioNet 2022. The plain physionet.org download links returned server errors for us, so use either of these:
   - Shivansh's downloader, which uses PhysioNet's official mirror and checks every file. It needs `uv` or the AWS command-line tool, but no account: `python -m cardio.data.download_unseen`.
   - The ZIP from the dataset page in your browser. Unzip it so that `data/raw/circor/training_data/13918_AV.wav` exists. If you get `training_data/training_data/`, move the files up one level.
5. Build Shivansh's PhysioNet manifest and caches (about 5 minutes). The second line must print nothing.

```
python -m cardio.data.build_unseen --dataset circor
git diff --stat data/manifests/circor_manifest.csv
```

6. Run `python -m pytest tests/` again. The PhysioNet checks now run as well.

7. Task 1: Secondary characteristics (`cardio/model/features.py`)

Goal. Measure simple things about every heartbeat recording. These numbers appear in the report next to the CNN's answer, and they feed your feature vote.

Steps.

1. Write one function, `characteristics(path)`, that returns a dictionary of numbers for one recording. The starter code below does exactly this.
2. Add a `main()` that runs it over a list of recordings and writes one CSV row each. It has three modes:
   - `--dataset yaseen` reads `data/manifests/yaseen_manifest.csv` and keeps its `patient_id`, `fold` and `label` columns;
   - `--dataset physionet` reads `data/manifests/circor_manifest.csv` and keeps `patient_id` and `path`;
   - `--folder <path>` takes every WAV in a folder, so Aviral can run it on the clinic recordings.
3. Run it for Yaseen and PhysioNet.

The 17 numbers per recording.

From	Numbers
`band_power_pct`	4: `band_20_100`, `band_100_250`, `band_250_500`, `band_500_800` (percent of power in each band)
Single values	`spectral_centroid_hz`, `murmur_duration_pct`, `murmur_intensity_db`, `systole_ms`, `diastole_ms`, `s1_s2_ratio_db`, `systolic_ratio`, `diastolic_ratio`
Yes or no, as 0 or 1	`murmur_detected`, `click_suspected`, `segmentation_ok`
The heart-rate recipe	`heart_rate_bpm`, `regularity`

`murmur_phase` is text, so leave it out. Several values are empty when segmentation fails. That is correct: do not fill them here, because Task 2 fills them with training-fold medians.

Starter code. The heart-rate recipe autocorrelates the heartbeat envelope and finds the strongest repeat between 0.33 and 1.5 seconds, which covers 40 to 180 beats per minute.

```
import numpy as np
from core.descriptors import _envelope, audio_descriptors
from core.features import CARDIO_CONFIG, load_wav

FS = CARDIO_CONFIG.sample_rate          # 8,000 samples per second

def heart_rate(x16, fs=FS, rate=100):
    """Heart rate in beats per minute, and regularity from 0 to 1."""
    e = _envelope(x16.astype(float), fs)[:: fs // rate]   # 100 values per second is plenty
    e = e - e.mean()
    ac = np.correlate(e, e, mode="full")[len(e) - 1:]
    ac = ac / (ac[0] + 1e-12)
    lo, hi = int(0.33 * rate), min(int(1.5 * rate), len(e) // 2)  # need at least two beats
    if hi <= lo:
        return None, None
    lag = lo + int(np.argmax(ac[lo:hi]))
    return 60.0 * rate / lag, float(ac[lag])

def characteristics(path):
    x = load_wav(str(path), CARDIO_CONFIG)                 # mono, 8 kHz, from any input rate
    x16 = (np.clip(x, -1, 1) * 32767).astype(np.int16)
    d = audio_descriptors(x16, fs=FS)
    row = {f"band_{k}": v for k, v in d["band_power_pct"].items()}
    for k in ("spectral_centroid_hz", "murmur_duration_pct", "murmur_intensity_db", "systole_ms",
              "diastole_ms", "s1_s2_ratio_db", "systolic_ratio", "diastolic_ratio"):
        row[k] = d.get(k)
    for k in ("murmur_detected", "click_suspected", "segmentation_ok"):
        row[k] = int(bool(d.get(k)))
    row["heart_rate_bpm"], row["regularity"] = heart_rate(x16)
    return row
```

Check that it worked.

- 1,000 Yaseen rows. That includes the 10 short clips the CNN skips; Task 5 joins on `patient_id`, so they simply drop out there.
- `segmentation_ok` is 1 for about 98% of clips.
- A heart rate for every clip, with a median of about 74 bpm and most clips between 60 and 90. We tested this recipe on fold 0: it agreed within 15% with the heart rate from segmentation, 60000 / (`systole_ms` + `diastole_ms`), for about three quarters of clips.
- Speed: about 14 ms per Yaseen clip and about 50 ms per PhysioNet recording, including loading the file. All of PhysioNet takes about 3 minutes.

8. Task 2: Feature vote (`cardio/fusion/votes.py`)

Goal. A logistic regression that reads the 17 numbers and gives the probability that a recording is abnormal, p_{LR}.

Why. It is the explainable second opinion. Its AUROC on its own shows how much the simple measurements know.

Steps.

1. Add an empty `cardio/fusion/__init__.py`, so `python -m cardio.fusion.votes` works.
2. Read `reports/heart_features_yaseen.csv`. The target is 1 if `label` is not 0 (AS, MS, MR or MVP) and 0 if it is 0 (Normal).
3. For each fold k from 0 to 4, fit on the other four folds and predict fold k. The pipeline below learns the filling and scaling from the training folds only.

4. Save `reports/heart_plr_yaseen.csv` (`patient_id`, `fold`, `label`, `p_lr`). Print the AUROC with its interval.
5. Fit once more on all Yaseen clips. That model is the feature vote used on PhysioNet (Task 5) and in the tool (Task 7).

Starter code.

```
import numpy as np, pandas as pd
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from cardio.eval.intervals import metric_intervals

def new_vote():
    return make_pipeline(SimpleImputer(strategy="median"), StandardScaler(),
                         LogisticRegression(class_weight="balanced", max_iter=1000))

df = pd.read_csv("reports/heart_features_yaseen.csv")
X = df.drop(columns=["patient_id", "fold", "label"]).to_numpy(dtype=float)
y = (df.label != 0).astype(int).to_numpy()
p_lr = np.zeros(len(df))
for k in range(5):
    train, test = (df.fold != k).to_numpy(), (df.fold == k).to_numpy()
    p_lr[test] = new_vote().fit(X[train], y[train]).predict_proba(X[test])[:, 1]
print(metric_intervals(y, p_lr, df.patient_id.to_numpy(), ["auroc"]))
```

Check that it worked. Every clip has a `p_lr`, and the AUROC is well above 0.5. Report whatever it is: a weak vote is a valid result, and fusion will simply give it a small weight.

9. Task 3: Metadata vote (also in `votes.py`)

Goal. A logistic regression on the patient's details that gives the probability of a murmur, p_M.

Why. It is the third vote in proposal 4.2. It must stay a small helper, with a weight of at most 0.3, never the decider.

Steps.

1. Build a table with one row per `patient_id` from `data/manifests/circor_manifest.csv`, using the starter code below:
   - the label is 1 (Present) if any visit says Present, and 0 if the visits that are known all say Absent; a patient whose visits are all Unknown is dropped;
   - age group as a number: Neonate 0, Infant 1, Child 2, Adolescent 3, Young adult 4; "nan" means missing;
   - sex: Female 0, Male 1; pregnancy: False 0, True 1; height and weight as numbers.
2. Make 5 folds over patients: `StratifiedKFold(n_splits=5, shuffle=True, random_state=0)` on the label. Save them as `cardio/data/circor_folds.csv` (`patient_id`, `fold`) and commit it, because Task 5 and Shivansh use the same folds.
3. Get out-of-fold p_M with the same `new_vote()` pipeline as Task 2. Save `reports/heart_pm_physionet.csv` and print the AUROC with its interval.

Starter code for step 1.

```
import json, numpy as np, pandas as pd

AGE = {"Neonate": 0, "Infant": 1, "Child": 2, "Adolescent": 3, "Young adult": 4, "Young Adult": 4}
num = lambda v: float(v) if v not in (None, "", "nan") else np.nan

c = pd.read_csv("data/manifests/circor_manifest.csv")
c["murmur"] = c["labels"].map(lambda s: json.loads(s)["murmur"])
meta = c["metadata"].map(json.loads)
c["age"] = meta.map(lambda d: AGE.get(d.get("Age"), np.nan))
c["sex"] = meta.map(lambda d: {"Female": 0, "Male": 1}.get(d.get("Sex"), np.nan))
c["height"] = meta.map(lambda d: num(d.get("Height")))
c["weight"] = meta.map(lambda d: num(d.get("Weight")))
c["pregnant"] = meta.map(lambda d: {"False": 0, "True": 1}.get(d.get("Pregnancy status"), np.nan))

def patient_label(visits):
    known = set(visits) - {"Unknown"}
    return np.nan if not known else int("Present" in known)

patients = c.groupby("patient_id").agg(
    label=("murmur", patient_label), age=("age", "first"), sex=("sex", "first"),
    height=("height", "first"), weight=("weight", "first"), pregnant=("pregnant", "first"),
).dropna(subset=["label"])
```

Check that it worked.

- 816 patients: 161 Present and 655 Absent.
- About 9% of ages and 12% of heights and weights are missing. Median filling inside `new_vote()` handles them.
- Expect a modest AUROC, above 0.5. Patient details are a weak signal, which is exactly why the weight is capped.

Watch out. Only these five facts are inputs. Never use the murmur grading, timing, pitch, shape or locations, the campaign, or the recording location: they give away the answer or describe how the data were collected.

10. Task 4: Calibration (`cardio/fusion/calibrate.py`)

Goal. One number, T, that makes the CNN's probabilities honest.

Why. Later steps compare probabilities with each other and with costs, so "80% sure" has to mean 80%.

Steps.

1. Read `reports/heart_oof_yaseen.csv`. Take the five `p_` columns, divide each row by its sum (the file rounds to 6 decimals), and use `label` as the target.
2. Temperature scaling works on log-probabilities: z = log p, and the calibrated probabilities are softmax(z / T). Find the T that minimises the negative log-likelihood (starter code below).
3. Measure the improvement honestly. For each fold k, fit T on the other four folds and apply it to fold k. Then compare, before and after, with intervals from `metric_intervals(..., ["ece"])`:
   - the five-class calibration error;
   - the refer-or-not calibration error, using 1 − p(Normal) as the probability and abnormal as the target.
4. Fit T once on all 990 clips. That final T is the one everyone uses. Save `reports/heart_calibration.json` with T per fold, the final T, and the errors before and after with intervals.

Starter code.

```
import numpy as np, pandas as pd
from scipy.optimize import minimize_scalar

def softmax_t(z, T):
    s = z / T
    s = s - s.max(axis=1, keepdims=True)
    q = np.exp(s)
    return q / q.sum(axis=1, keepdims=True)

def fit_temperature(P, y):
    z = np.log(np.clip(P, 1e-12, 1))
    nll = lambda T: -np.mean(np.log(softmax_t(z, T)[np.arange(len(y)), y] + 1e-12))
    return minimize_scalar(nll, bounds=(0.05, 10), method="bounded").x

oof = pd.read_csv("reports/heart_oof_yaseen.csv")
P = oof.filter(like="p_").to_numpy()
P = P / P.sum(axis=1, keepdims=True)
y = oof.label.to_numpy()
```

Check that it worked.

We ran this starter code to check it. You should see roughly the same:

- T below 1: about 0.5 for every fold (0.45 to 0.53), and 0.50 on all 990 clips;
- five-class calibration error falling from 0.058 to about 0.006, well under the 0.05 target;
- refer-or-not calibration error falling from 0.020 to about 0.002;
- accuracy unchanged at 99.29%, because a temperature never changes which class wins.

11. Task 5: Late fusion and the two comparisons (`cardio/fusion/late_fusion.py`)

Goal. Combine the votes into $p_h = w_1 \cdot p_{CNN} + w_2 \cdot p_{LR} + w_3 \cdot p_M$, and show whether it helps.

Step 1: CNN probabilities for PhysioNet. Shivansh's `run_unseen.py` will give these. Until it exists, this code writes them from the cached spectrograms in a few minutes (or ask Aviral for the file):

```
import csv, numpy as np, pandas as pd
from cardio.model.final_model import CLASSES, FinalModel

model, rows = FinalModel(), []
with open("data/manifests/circor_manifest.csv", encoding="utf-8") as fh:
    for r in csv.DictReader(fh):
        with np.load(r["cache_path"]) as cache:
            feats = cache["features"]
        if len(feats) == 0:
            continue
        p = np.asarray(model.float_model(model.normalise(feats), training=False))
        p = p.reshape(len(feats), -1).mean(axis=0)
        rows.append({"patient_id": r["patient_id"], "path": r["path"],
                     **{f"p_{k}": v for k, v in zip(CLASSES, p)}})
pd.DataFrame(rows).to_csv("reports/heart_cnn_physionet.csv", index=False)
```

Apply your T from Task 4 to these probabilities, then take $p_{CNN} = 1 - p(\text{Normal})$. Average per patient over their recordings, and do the same for your PhysioNet characteristics.

Step 2: the fusion itself.

- The weights come from a grid in steps of 0.05 that add to 1, with w_3 at most 0.3.
- Pick the weights with the lowest log-loss on the training folds, then apply them to the test fold.
- If a vote is missing for a case (no patient details, for example), give it weight 0 and divide the other weights by their sum.
- Return each result as `FusedOutput("heart", p_abnormal, votes, weights)` from `core/contracts.py`. It refuses w_3 above 0.3 for you.

```
import numpy as np
from sklearn.metrics import log_loss

def weight_grid(use_metadata, step=0.05, w3_max=0.3):
    n = round(1 / step)
    for i in range(n + 1):
        for k in range(n + 1 - i):
            w1, w3 = i * step, k * step
            if w3 > w3_max + 1e-9 or (k and not use_metadata):
                continue
            yield round(w1, 2), round(1 - w1 - w3, 2), round(w3, 2)

def best_weights(y, p_cnn, p_lr, p_m=None):
    def loss(w):
        p = w[0] * p_cnn + w[1] * p_lr + (w[2] * p_m if p_m is not None else 0)
        return log_loss(y, np.clip(p, 1e-6, 1 - 1e-6))
    return min(weight_grid(p_m is not None), key=loss)
```

Step 3: Comparison A, generalisation. This uses no labels from the new dataset to fit anything.

- Choose (w_1, w_2) on Yaseen, using the out-of-fold p_{CNN} (after T) and p_{LR}.
- Apply them unchanged to PhysioNet, and to BMD-HS once Shivansh shares it, with the feature vote trained on all of Yaseen. On PhysioNet the target is the patient label from Task 3 (Present = 1).
- Compare the plain CNN with CNN plus feature vote, with intervals:
  - refer-or-not macro-F1 at 0.5 (the target is a gain of at least 3 points);
  - AUROC.

Step 4: Comparison B, metadata gain. This runs inside PhysioNet, using the folds in `cardio/data/circor_folds.csv`.

- CNN plus feature vote, with the weights chosen on the training folds.
- CNN plus feature vote plus metadata vote, chosen the same way, with w_3 at most 0.3.
- Compare the referral AUROC, with intervals. The target is a gain of at least 0.02.

Step 5. Save both comparisons in `reports/heart_fusion.json`: the weights chosen (and per fold for Comparison B), and every metric with its interval.

What to expect. On Yaseen the CNN is already perfect at refer-or-not (AUROC 1.000), so the Yaseen weights will lean almost entirely on the CNN. That is a real result; report it. Comparison B shows whether patient details add anything beyond the recording. With w_3 capped at 0.3 the gain may be small, and either outcome is a result.

12. Task 6: Add the adapted model (roadmap S4)

Goal. Put the full CardioNary heart branch into Comparison A: the adapted CNN plus the votes.

Steps.

1. Split the PhysioNet patients the way Aviral's `cardio/eval/requantise.py` does: random halves with seed 0. 64 recordings from one half adapt the model, the other half is measured, and then the halves swap.
2. Adapt with `adapt_bn(windows, model)` from `cardio/eval/real_world.py`. Compute the adapted p_{CNN} for the measured half, applying T as well.
3. Fuse with the same Yaseen weights. Add three rows to `reports/heart_fusion.json`, each with intervals:
   - plain CNN;
   - adapted CNN;
   - adapted CNN plus feature vote.

Shivansh's `cardio/tta/bn_adapt.py` will replace `adapt_bn` when it lands; the results should not change.

13. Task 7: Hand-off for the tool, and `cardio/run_all.py` (roadmap S5)

1. Save the fitted numbers the tool needs in `reports/heart_fusion_params.json`, as plain numbers rather than pickle files:
   - T;
   - the Yaseen weights (w_1, w_2) and the PhysioNet weights with metadata (w_1, w_2, w_3);
   - for each vote: the column order, the filling medians, the scaler means and scales, and the logistic-regression coefficients and intercept.
2. Write one function, `heart_fusion(p_cnn, features, metadata)`, that uses only that file and returns a `FusedOutput`. Aviral's tool page and edge budget will call it.
3. Write `cardio/run_all.py`. It runs every heart script in this order and stops at the first failure. The last two lines run only when `--clinic-dir` is given, which happens on Aviral's laptop.

```
python -m cardio.model.cv_oof --from-oof
python -m cardio.eval.agreement --manifest data/manifests/circor_manifest.csv --name physionet_2022
python -m cardio.model.features --dataset yaseen
python -m cardio.model.features --dataset physionet
python -m cardio.fusion.votes
python -m cardio.fusion.calibrate
python -m cardio.fusion.late_fusion
python -m cardio.eval.requantise --site physionet_2022
python -m cardio.eval.edge_budget
python -m cardio.eval.real_world --data-dir <clinic folder> --adapt
python -m cardio.eval.requantise --site clinic --data-dir <clinic folder>
```

Add Shivansh's scripts (unseen datasets, adaptation, noise tests) as they land.

14. Task 8: Your report sections (roadmap S6)

You write Section 4.2 (characteristics, votes, late fusion) and the calibration part:

- the method, a few lines per part (the wording of this guide is a good start);
- result tables with 95% intervals:
  - feature-vote AUROC and metadata-vote AUROC;
  - T, and the calibration errors before and after;
  - Comparison A and Comparison B;
- what did not work, for example the weights leaning on the CNN, or a weak metadata vote.

Also write a short note per task in `cardio/experiments/`, failures included, and your part of the AI-usage note: which tools, for what, and how you checked their output.

15. Rules, git routine and who to ask

Rules that keep the numbers honest.

- Fit filling, scaling, temperature and weights on training folds only, and never score a fold you fitted on.
- PhysioNet folds go by `patient_id` from the manifest, never by recording.
- Every number in the report comes from a script and a committed result file. Seeds are fixed at 0.
- The clinic recordings (`Real_World_Data`) never come to your laptop. If your features are needed on them, Aviral runs your script there.

Git routine.

- Work on `cardio-anshuman-<topic>` branches made from `cardio`. Commit small steps, push, and open a pull request into `cardio` that one teammate approves; merge with "Create a merge commit".

- Each morning, bring in the team's latest work: `git checkout cardio`, `git pull`, `git checkout <your branch>`, `git merge cardio`.

Who to ask.

Question about	Ask
The out-of-fold file, the final model, `FinalModel`, intervals, `adapt_bn`, the tool and edge budget	Aviral
The PhysioNet and BMD-HS manifests and caches, `metrics.py`, the BMD-HS data	Shivansh